

合肥工业大学

数论课程设计



设计题目 数论函数的反演在降低算法时间复杂度上的应用

学生姓名 张致远

学 号 2023213980

专业班级 信息与计算科学 23-1 班

2024 年 6 月 11 日

合肥工业大学课程设计任务书

设计 题目	数论函数的反演在降低算法时间复杂度上的应用	成绩	
课 程 设 计 主 要 内 容	此篇数论报告旨在挖掘抽象的数论知识在实际过程中的应用。通常我们在编写算法处理问题时，当遇到过大的数据量时，平常的算法时间复杂度过高，从而大大降低程序运行效率。然而对于特殊的问题，我们可以通过使用数论函数的反演来降低算法的时间复杂度，从而优化算法。本篇从较为简单的欧拉反演引入，进而讨论莫比乌斯反演的具体应用。为了使抽象的数论知识落到实际应用上，在公式推导之外，附加了典型问题和具体的代码实现。		

目录

一、简单的欧拉反演的应用	1
二、莫比乌斯反演的历史起源	2
三、莫比乌斯反演的性质与证明	2
四、预备知识	3
五、具体例题的应用	4

一、简单的欧拉反演的应用

欧拉反演不怎么实用，但在做某些题时要比莫比乌斯反演要简便。但是对于一些求权值和的反演题比较合适。我们先由简单的欧拉反演来引入数论函数反演优化时间复杂度的问题。

欧拉反演的核心公式：

$$n = \sum_{d|n} \varphi(d)$$

证明如下：

$$n = \sum_{d|n} \sum_{i=1}^n [\gcd(i, n) = d]$$

$$n = \sum_{d|n} \sum_{i=1}^{\frac{n}{d}} \left[\gcd(i, \frac{n}{d}) = d \right]$$

$$n = \sum_{d|n} \varphi\left(\frac{n}{d}\right)$$

又因为一个数的约数是成对出现的

$$n = \sum_{d|n} \varphi(d)$$

知道了核心公式以后怎么进行反演呢？欧拉反演的原理就是把一些式子里求的值，强行换成 $\sum_{d|x} \varphi(d)$ 的形式，其中 x 就是要求的值。

具体怎么实现，让我们来看一个例子：求 $\sum_{i=1}^n \gcd(i, n)$

求的是所有最大公因数的值，那么我们可以把 $\gcd(i, n)$ 看成是公式里的 n ，反演后就是

$$\sum_{i=1}^n \sum_{d|\gcd(i, n)} \varphi(d)$$

继续推导容易得到

$$\begin{aligned} &= \sum_{i=1}^n \sum_{d|i} \sum_{d|n} \varphi(d) \\ &= \sum_{d|n} \sum_{i=1}^n \sum_{d|i} \varphi(d) \end{aligned}$$

对于每个 d ，在 n 的范围内只对 $\frac{n}{d}$ 个数有贡献，因此得到

$$\sum_{d|n} \frac{n}{d} \varphi(d)$$

这样便只需要遍历 n 的约数 d ，时间复杂度降到了 $O(\sqrt{n})$

二、莫比乌斯反演的历史起源

下面让我们来追溯一下莫比乌斯反演的历史起源。

莫比乌斯反演的历史起源涉及多个领域的发展，包括数论、拓扑学和算法学等。

莫比乌斯反演的历史起源和发展是多方面的，它不仅在数学中有深厚的根基，而且在其他科学领域也有着广泛的应用。这一概念的命名来源于德国数学家奥古斯特·费迪南德·莫比乌斯，他在 19 世纪对这一概念进行了深入的研究和系统化的发展。然而，莫比乌斯反演的思想可以追溯到更早的数学家，特别是欧拉的工作。

在数论方面，莫比乌斯反演与算术函数和它们的和函数之间的关系密切相关。例如，莫比乌斯函数 $\mu(n)$ 是一个积性函数，它在数论中扮演着重要的角色。对于该方面的性质与证明在本篇下一部分会给出。

在拓扑学中，莫比乌斯反演的概念可以通过实验来形象地说明。例如，通过扭转并粘合一条纸带的两端，可以得到一个具有特殊拓扑性质的曲面，即莫比乌斯带。在这个曲面上，沿着特定的闭路绕一圈后，铅笔尖的方向会与最初的方向相反，这展示了莫比乌斯带与普通圆柱面不同的拓扑性质。

在算法学中，莫比乌斯反演通常涉及到两个求和符号，它与最大公约数有关的问题紧密相关。通过枚举 $\gcd$ ，并使用积性函数预处理和分块技术，可以在 $O(n)$ 的时间复杂度内解决问题。

总的来说，莫比乌斯反演的历史起源是复杂且跨学科的，它不仅在数学领域有着悠久的历史和应用，在其他科学领域也有着广泛的影响。

在本篇的下半部分，将会具体讨论莫比乌斯反演在算法学中的应用。

三、莫比乌斯反演的性质与证明

首先我们来看莫比乌斯函数。

$$\mu(n) = \begin{cases} 1 & n = 1 \\ 0 & n \text{ 含有平方因子,} \\ (-1)^k & k \text{ 为 } n \text{ 的不同质因子的个数.} \end{cases}$$

下面来看莫比乌斯函数的性质：

- 莫比乌斯函数是积性函数。
- 莫比乌斯函数有以下性质：

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & n = 1 \\ 0 & n \neq 1 \end{cases}$$

下面我们来证明第二条性质：设 $n = \prod_{i=1}^k p_i^{c_i}, n' = \prod_{i=1}^k p_i$

那么 $\sum_{d|n} \mu(d) = \sum_{d|n'} \mu(d) = \sum_{i=0}^k \binom{k}{i} \cdot (-1)^i = (1 + (-1))^k$ 根据二项式定理，易知该式子的值在 $k = 0$ 即 $n = 1$ 时值为 1 否则为 0。

莫比乌斯反演：

设 $f(n), g(n)$ 为两个数论函数。

如果有 $f(n) = \sum_{n|d} g(d)$, 那么有 $g(n) = \sum_{n|d} \mu(\frac{d}{n}) f(d)$ 。

证明：

$$\sum_{d|n} \mu(d) f(\frac{n}{d}) = \sum_{d|n} \mu(d) \sum_{k|\frac{n}{d}} g(k) = \sum_{k|n} g(k) \sum_{d|\frac{n}{k}} \mu(d) = g(n)$$

简单来说，就是用 $\sum_{d|n} g(d)$ 来替换 $f(\frac{n}{d})$, 再变换求和顺序。

四、预备知识

在讨论具体问题之前，还有几个待解决的问题。其中最重要的问题是如何求莫比乌斯函数？

由前面讨论的性质我们得知，莫比乌斯函数是积性函数，因此可以采用线性筛的方法来求解。(尽管方法不同，但是线性筛可以用来求解大多数的积性函数。因为此篇报告为数论报告，因此不过多讨论，直接附上 C++ 代码)

```
void getMu() {
    mu[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (!flg[i]) p[++tot] = i, mu[i] = -1;
        for (int j = 1; j <= tot && i * p[j] <= n; ++j) {
            flg[i * p[j]] = 1;
            if (i % p[j] == 0) {
                mu[i * p[j]] = 0;
                break;
            }
            mu[i * p[j]] = -mu[i];
        }
    }
}
```

另一问题是整除分块，也叫数论分块。在此篇只做简单介绍。

不难发现 $\lfloor \frac{n}{i} \rfloor$ 在一段区间里是连续的，所以我们可以直接计算出每个区间的左右端点。(附 C++ 代码如下)

```
long long division_block(long long n){
    long long res = 0;
    for(long long l = 1, r; l <= n; l = r + 1){
        r = n / (n / l);
        res += n / l * (r - l + 1);
    }
    return res;
}
```

五、具体例题的应用

我们已经有了足够的预备知识，下面让我们来看具体的莫比乌斯反演问题，体会其在优化时间复杂度上的作用。

问题描述：

给定 n , 求 $\sum_{i=1}^n \sum_{j=i+1}^n \gcd(i, j)$

推导:

$$\begin{aligned}
 & \sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) \\
 &= \sum_{k=1}^n k \sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) = k \\
 &= \sum_{k=1}^n k \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{d|\gcd(i, j)} \mu(d) \\
 &= \sum_{k=1}^n k \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{n}{k} \rfloor} \gcd(i, j) = 1 \\
 &= \sum_{k=1}^n k \sum_{d=1}^n \mu(d) \left(\lfloor \frac{n}{kd} \rfloor \right)^2
 \end{aligned}$$

令 $T = kd$

$$\begin{aligned}
 &= \sum_{T=1}^n \left(\lfloor \frac{n}{T} \rfloor \right)^2 \sum_{k|T} k \mu\left(\frac{T}{k}\right) \\
 &= \sum_{T=1}^n \left(\lfloor \frac{n}{T} \rfloor \right)^2 \varphi(T)
 \end{aligned}$$

$\varphi(T)$ 可以线性筛, $\left(\lfloor \frac{n}{T} \rfloor\right)^2$ 可以整除分块。

由此, 我们得到的时间复杂度为: 预处理 $O(n)$, 查询 $O(\sqrt{n})$, 因此, 总的时间复杂度为 $O(n\sqrt{n})$ 。

下附 C++ 代码:

```

#include <bits/stdc++.h>
using namespace std;
#define N 2000005
#define ll long long
int n, prm[N], phi[N]; ll ans, sPhi[N]; bool vs[N];
void sieve(int x)
{
    vs[1]=phi[1]=sPhi[1]=1;
    for(int i=2; i<=x; ++i)
    {
        if(!vs[i]) prm[++prm[0]]=i, phi[i]=i-1;
    }
}

```



```

        for(int j=1;i*prm[j]<=x;++j)
        {
            vs[i*prm[j]]=1;if(!(i%prm[j])) {phi[i*prm[j]]=phi[i]*prm[j];break;}
            phi[i*prm[j]]=phi[i]*(prm[j]-1);
        }
        sPhi[i]=sPhi[i-1]+phi[i];
    }
}

int main()
{
    scanf("%d",&n);sieve(n);
    for(int i=1,t;i<=n;i=t+1) t=n/(n/i),ans+=1ll*(sPhi[t]-sPhi[i-1])*(n/i)*(n/i);
    printf("%lld\n",ans-(1ll*n*(n+1)>>1)>>1);
}

```